

Grounded Agents: Industrial Structure and the Robotics Transition

Application Paper I of the *Grounded Agents* series

A Companion to the Theory Paper

Macharia Bari

Abstract

We apply the inference-first framework of the companion theory paper to the industrial structure that produces and deploys artificial agents. Grounding is environment-indexed, non-derivable from public corpora, and non-purchasable; the closed epistemic loop is the proprietary asset. We prove that generative components (model, orchestration) commoditize without changing the grounding coefficient G , that the integrator who closes the loop acquires a structurally proprietary resource, that integration cost decays slower than component cost, and that the robotics transition differs structurally from the PC transition because robots internalize grounding while PCs externalize it. We introduce the tinkering threshold, derive staggered domain-specific eras, and predict a landscape of domain-specific integrators rather than a single general winner. All results are conditional on the axioms of Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*.

Remark 0.1 (Reading). Theorem numbers in this paper are local. The framework, definitions, and proofs of the results recalled in Section 2 live in Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*; they are stated here without proof for readability.

Contents

1	Introduction	2
2	Recalled Framework	2
3	Components and Integrators	4
3.1	The Commoditization Mechanism	5
3.2	Accidental Fitness and Outsider Dominance	5
4	Environment-Indexed Grounding	7
5	Loop Closure and the Grounded Data Flywheel	7
6	The Integrator Moat	8
7	Predictions for Industrial Structure	8
7.1	Failure Modes	8
8	The Integrator Position	9

9	The Tinkering Threshold and the PC-Era Analogy	9
9.1	Cost Curves	9
9.2	The Decay Divergence	10
9.3	Grounding Externalization and Internalization	10
9.4	Multi-Environment Generality	11
9.5	Predictions	11
9.6	How the Robotics Trajectory Differs from the PC Trajectory	12
10	Open Problems	12
11	Conclusion	13
A	Summary of Definitions (Application-Specific and Imported)	14
B	Summary of Theorems (Application-Specific and Imported)	15
C	Industrial-Structure Level Table	15
D	PC-Era vs. Robotics-Era Comparison	16
E	A Note on Method	16

1 Introduction

In the theory paper, agents are characterized by the triple (R, D, G) : model-reach, recursion depth, and the grounding coefficient. The industrial question this paper addresses is structural: who produces these agents, where does value accrue in doing so, and when does the production of grounded agents become cheap enough to transform an economy?

Three questions organize the paper. First: in the present technological moment, the capacities that constitute an agent—model-generation, orchestration, sensing, actuation—are produced by different firms and exchanged as components. If generative components commoditize, where does value accrue? Second: is grounding a global property of an agent, or a property of an agent in an environment? Third: when does a closed-loop system become affordable by an individual? Each question receives a formal answer, and the three answers jointly determine the shape of the coming industry.

The paper proceeds as follows. Section 2 recalls the framework. Section 3 formalizes components and the commoditization mechanism, including the mechanism by which outsiders dominate adjacent component layers. Section 4 makes grounding environment-indexed. Section 5 derives the grounded-data flywheel. Section 6 proves the integrator moat. Section 7 collects structural predictions and records failure modes. Section 9 formalizes the tinkering threshold and shows why the robotics transition differs from the PC transition. Section 10 records open problems, and Section 11 concludes.

2 Recalled Framework

This paper applies the framework of Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*. For convenience we recall the definitions and results used below; proofs are omitted here and given in full in the theory paper.

Definition 2.1 (Consequence System). A *consequence system* is a pair $\mathfrak{S} = (\mathcal{C}, \vdash)$ satisfying, for all $\Gamma, \Delta \subseteq \mathcal{C}$ and $c \in \mathcal{C}$:

(C1) **Reflexivity.** If $c \in \Gamma$, then $\Gamma \vdash c$.

(C2) **Monotonicity.** If $\Gamma \vdash c$ and $\Gamma \subseteq \Delta$, then $\Delta \vdash c$.

(C3) **Cut.** If $\Gamma \vdash c$ and $\Delta \cup \{c\} \vdash d$, then $\Gamma \cup \Delta \vdash d$.

Definition 2.2 (Closure Operator). Let $\mathfrak{S} = (\mathcal{C}, \vdash)$ be a consequence system. A map $\sigma : \mathcal{P}(\mathcal{C}) \rightarrow \mathcal{P}(\mathcal{C})$ is a *closure operator* for $\mathfrak{S}$ if it is extensive ($\Gamma \subseteq \sigma(\Gamma)$), idempotent ($\sigma(\sigma(\Gamma)) = \sigma(\Gamma)$), and monotone. We call $\sigma(\Gamma)$ the *inferential closure* of Γ .

Definition 2.3 (Agent). An *agent* is a tuple $\mathcal{A} = (\mathcal{I}, \mathcal{S}, \mu)$ where $\mathcal{I} = (\mathcal{C}, \vdash, \sigma)$ is an inference system, $\mathcal{S}$ is a set of *sensory states*, and $\mu : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{C})$ is a *grounding map*. The *belief set* at sensory state s is $\mathcal{B}_{\mathcal{A}}(s) = \sigma(\mu(s))$.

Definition 2.4 (Grounding Coefficient). Let $\mathcal{A}$ be an agent with finite belief sets. For each $s \in \mathcal{S}$, define

$$G(\mathcal{A}, s) = \frac{|\{c \in \mathcal{B}_{\mathcal{A}}(s) : c \text{ is grounded in } \mathcal{A}'\text{'s own sensory history}\}|}{|\mathcal{B}_{\mathcal{A}}(s)|}.$$

The *grounding coefficient* of $\mathcal{A}$ is $G(\mathcal{A}) = \inf_{s \in \mathcal{S}} G(\mathcal{A}, s)$.

Definition 2.5 (Model). A *model* for agent $\mathcal{A}$ is a subset $M \subseteq \mathcal{C}$ such that M is consistent and $\sigma(M) = M$.

Definition 2.6 (Model-Generator). Agent $\mathcal{A}$ has a *model-generator* if there exists a function $\gamma : \mathcal{P}(\mathcal{C}) \rightarrow \mathcal{P}(\mathcal{C})$ such that for every $s \in \mathcal{S}$: (M1) $\gamma(\mu(s))$ is a model; (M2) if the target set $T \neq \emptyset$, then $\gamma(\mu(s)) \cap T \neq \emptyset$; (M3) γ is extensive; (M4) γ is monotone.

Definition 2.7 (Model-Reach). The *model-reach* of γ is

$$R(\gamma) = \sup_{s \in \mathcal{S}} |\gamma(\mu(s)) \setminus \mu(s)|.$$

Definition 2.8 (Observation Operator). An *observation operator* for agent $\mathcal{A}$ is a function

$$\omega : \mathcal{P}(\mathcal{C}) \times \mathcal{S} \rightarrow \{\top, \perp\}$$

such that $\omega(M, s) = \top$ implies $M \cup \mu(s)$ is consistent, and $\omega(M, s) = \perp$ implies $M \cup \mu(s)$ is inconsistent.

Definition 2.9 (Epistemic Loop). The *epistemic loop* of $\mathcal{A}$ is the iteration

$$M_{n+1} = \begin{cases} \gamma(M_n \cup \mu(s_n)), & \text{if } \omega(\gamma(M_n \cup \mu(s_n)), s_{n+1}) = \top, \\ M_n, & \text{otherwise.} \end{cases}$$

Theorem 2.10 (Convergence). Suppose $\mathcal{C}$ is finite and γ is extensive and monotone. Then the epistemic loop converges to a fixed point in finitely many steps.

Definition 2.11 (Cognition Level). The *cognition level* of agent $\mathcal{A}$ is the pair $(R(\gamma), D(\gamma))$, where D is *recursion depth* (the depth of iterative self-modeling). Within Level 5, the *refined level* $(R(\gamma), D(\gamma), G(\mathcal{A}))$ distinguishes: 5^{++} ($G = 1$), 5^+ ($0 < G < 1$), and 5^* ($G = 0$).

Theorem 2.12 (Grounding Gap). *If $G(\mathcal{A}) = 0$, then $\mathcal{A}$ has no self-sourced observation operator, and no content-generating operation can detect its own error.*

Theorem 2.13 (Scaling Does Not Close the Gap). *Increasing $R(\gamma)$ or $D(\gamma)$ in an ungrounded agent does not increase $G(\mathcal{A})$.*

Theorem 2.14 (Embodiment Closes the Loop). *An agent equipped with self-sourced sensory states, a self-sourced grounding map, and a self-sourced observation operator has $G > 0$, and its epistemic loop converges (whenever γ is extensive, monotone, and $\mathcal{C}$ finite).*

Remark 2.15. Throughout, “ungrounded” refers to $G = 0$, “grounded” to $G > 0$. Level 5^* denotes an ungrounded symbolic agent, 5^+ a partially grounded one, and 5^{++} a fully grounded one. All definitions and results in this section are from Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*.

3 Components and Integrators

The framework so far concerns agents considered in isolation. We now consider a structural question about how agents are produced. In the present technological moment, the capacities that constitute an agent—model-generation, orchestration, sensing, actuation—are produced by different firms and exchanged as components. We formalize this and derive consequences for where value accrues.

Definition 3.1 (Component). Let $\mathcal{A} = (\mathcal{I}, \mathcal{S}, \mu)$ be an agent with model-generator γ and closure operator σ . A *component* of $\mathcal{A}$ is a proper subsystem $\kappa \subseteq \mathcal{A}$. We distinguish:

- (a) a *generative component*, if $\kappa = \gamma$ or $\kappa = \sigma$;
- (b) a *transductive component*, if κ affects μ or ω .

Definition 3.2 (Substitution). A *substitution* of κ by κ' in $\mathcal{A}$ yields the agent $\mathcal{A}' = \mathcal{A}[\kappa \rightarrow \kappa']$ obtained by replacing κ with κ' and retaining all other subsystems.

Definition 3.3 (Commoditization). A class $\mathcal{K}$ of components is *commoditized* if for every $\kappa_1, \kappa_2 \in \mathcal{K}$ and every agent $\mathcal{A}$ containing κ_1 , the substitution $\mathcal{A}[\kappa_1 \rightarrow \kappa_2]$ has R and D within bounded variation of $R(\mathcal{A})$ and $D(\mathcal{A})$.

Theorem 3.4 (Generative Substitution Invariance of G). *If κ is a generative component of $\mathcal{A}$, then substitution of κ leaves $G(\mathcal{A})$ invariant.*

Proof. By Definition 2.4, $G(\mathcal{A})$ is determined by the *source* of the grounding map μ . Generative components γ and σ act on $\mathcal{P}(\mathcal{C})$; they do not produce μ . Hence substitution of γ or σ leaves μ unchanged, and therefore leaves $G(\mathcal{A})$ unchanged. $\square$

Corollary 3.5 (Model Layer Commoditizes). *Swapping model or orchestration components changes R and D but not G . Hence such substitution cannot move an agent across the $5^* \rightarrow 5^+$ threshold.*

Proof. Immediate from Theorem 3.4 and the refined level assignment. $\square$

This is the formal statement that scale and orchestration are changes of degree, not kind. An agent built on any of several interchangeable models is not thereby more grounded.

3.1 The Commoditization Mechanism

The preceding results establish that generative components commoditize. They do not explain *how* commoditization proceeds, nor where the dominant firms of the next component layer will come from. We now address both.

Definition 3.6 (Interface). Let $\mathcal{K}$ be a class of components. An *interface* for $\mathcal{K}$ is a set I of conventions—electrical, protocol, API, mechanical—by which any member of $\mathcal{K}$ can be invoked, connected, or substituted into a system.

Definition 3.7 (Interface Standard). A class $\mathcal{K}$ is *interface-standard* if all members of $\mathcal{K}$ satisfy a common interface I .

Definition 3.8 (Performance Parity). A class $\mathcal{K}$ exhibits *performance parity* if for all $\kappa_1, \kappa_2 \in \mathcal{K}$ and every agent $\mathcal{A}$ containing κ_1 , the substitution $\mathcal{A}[\kappa_1 \rightarrow \kappa_2]$ has R and D within bounded variation of $R(\mathcal{A})$ and $D(\mathcal{A})$.

Proposition 3.9 (Interfaces Drive Commoditization). *If a class $\mathcal{K}$ is interface-standard and exhibits performance parity, then $\mathcal{K}$ is commoditized in the sense of Definition 3.3.*

Proof. By Definition 3.7, any member of $\mathcal{K}$ satisfies the common interface. By Definition 3.8, substitution preserves R and D within bounded variation. Together these imply the substitution condition of Definition 3.3. Hence $\mathcal{K}$ is commoditized. $\square$

Corollary 3.10 (Spec Sheets Are Downstream). *Parameter counts, token windows, clock speeds, and comparable specification variables are determined by interface competition, not by it. The standardizing event is the interface, not the specification.*

Remark 3.11 (Historical Parallels). CPUs standardized on instruction-set architecture (x86, ARM), not on clock speed. Memory standardized on DIMM slots and DDR protocols, not on capacity. In both cases the interface standard preceded and drove commoditization, and the specification became a competitive variable within an already standardized class. The same pattern is now visible in model interfaces: OpenAI-compatible APIs, function-calling schemas, and multi-modal input formats are standardizing before model performance converges.

3.2 Accidental Fitness and Outsider Dominance

We now formalize the mechanism by which a firm that dominates one commodity specialization can come to dominate the component layer of an adjacent workload it never targeted. This is the mechanism by which Nvidia came to dominate AI compute: GPUs were designed for graphics, not for neural networks, and their architecture happened to fit matrix multiplication at scale.

Definition 3.12 (Workload Class). A *workload class* W is a class of computations or transductions that an agent or system must perform.

Definition 3.13 (Design Profile). For a component κ , the *design profile* $P(\kappa)$ is the set of workload classes for which κ was designed.

Definition 3.14 (Substrate Fitness). The *substrate fitness* $F(\kappa, W)$ of component κ for workload class W is a measure of how well κ performs W , independent of whether $W \in P(\kappa)$.

Definition 3.15 (Commodity Specialization). A component κ is a *commodity specialization* for workload W_0 if:

- (a) $P(\kappa) = \{W_0\}$,
- (b) κ is produced at industrial scale for W_0 ,
- (c) κ is available at low unit cost.

Definition 3.16 (Accidental Fit). A component κ has *accidental fitness* for workload W_1 if $W_1 \notin P(\kappa)$ and $F(\kappa, W_1)$ is high.

Theorem 3.17 (Accidental Fit Produces Outsider Dominance). *Let κ be a commodity specialization for workload W_0 with accidental fitness for workload W_1 . Let Φ_κ be the firm dominating production of κ . Then Φ_κ can dominate the component layer of W_1 .*

Proof. By Definition 3.15, κ is available at low unit cost. By Definition 3.16, $F(\kappa, W_1)$ is high. Consumers of W_1 therefore adopt κ because it is cheap and fit. Firms targeting W_1 directly face two options: (a) build a competing component at higher unit cost, because they lack Φ_κ 's scale in W_0 ; or (b) adopt κ and compete on integration, at which point Φ_κ retains the scale advantage. In either case, Φ_κ captures the W_1 component market. $\square$

Corollary 3.18 (Outsider Unpredictability). *Φ_κ is not predictable from the landscape of W_1 -targeted firms, because Φ_κ is not one of them.*

Corollary 3.19 (The Nvidia Case). *Graphics processing units were designed for $W_0 = \text{graphics rendering}$. They accidentally fit $W_1 = \text{matrix multiplication at scale, the core computation of neural network training}$. Nvidia, as the dominant producer of GPUs, captured the AI compute market without having designed for AI.*

Prediction 3.20 (The Future Nvidia-Like Firm). The dominant component-layer firm of the robotics era will not be a firm currently targeting robotics. It will be a firm dominating a commodity specialization whose product accidentally fits the robotics grounding loop.

Remark 3.21 (Candidates). Commodity specializations whose products may accidentally fit robotics grounding loops include:

- (a) smartphone SoCs, built for phones, potentially fitting edge inference;
- (b) smartphone camera modules, built at billions-per-year scale, potentially fitting visual perception;
- (c) MEMS inertial measurement units, built for phones, potentially fitting proprioception;
- (d) automotive sensors, built for ADAS, potentially fitting navigation;
- (e) haptic actuators, built for phones and controllers, potentially fitting tactile interaction.

Which of these produces the future Nvidia is an empirical question the framework does not settle. The framework predicts only that the firm will come from *one* of them, not from the robotics-targeted landscape.

4 Environment-Indexed Grounding

Grounding is not a global property of an agent. It is a property of an agent *in an environment*. We now make this explicit.

Definition 4.1 (Environment). An *environment* $\mathcal{E}$ is a structured domain of interaction with which an agent’s transductive components couple. Environments are *physical* if their states are produced by transductive coupling to matter, and *virtual* if their states are produced by software interaction.

Definition 4.2 (Environment-Indexed Grounding). Let $\mathcal{A}$ be an agent and $\mathcal{E}$ an environment. Define

$$G_{\mathcal{E}}(\mathcal{A}) = \inf_{s \in \mathcal{S}_{\mathcal{E}}} G(\mathcal{A}, s)$$

where $\mathcal{S}_{\mathcal{E}} \subseteq \mathcal{S}$ is the set of sensory states produced by interaction with $\mathcal{E}$.

Remark 4.3. When the environment is unrestricted, $G_{\mathcal{E}}(\mathcal{A})$ reduces to the global grounding coefficient $G(\mathcal{A})$ of Definition 2.4. The environment-indexed definition is thus a refinement, not a replacement.

Theorem 4.4 (Vertical Specificity). *For environments $\mathcal{E}_1 \neq \mathcal{E}_2$, $G_{\mathcal{E}_1}(\mathcal{A}) > 0$ does not imply $G_{\mathcal{E}_2}(\mathcal{A}) > 0$.*

Proof. $G_{\mathcal{E}}(\mathcal{A})$ is defined in terms of sensory states produced by interaction with $\mathcal{E}$. Such states are causally specific to $\mathcal{E}$. An agent that has interacted with $\mathcal{E}_1$ acquires grounding in $\mathcal{E}_1$; without interaction with $\mathcal{E}_2$, it has no self-sourced states in $\mathcal{S}_{\mathcal{E}_2}$, hence $G_{\mathcal{E}_2}(\mathcal{A}) = 0$. Partial transfer occurs only where $\mathcal{E}_1$ and $\mathcal{E}_2$ share sensory modality structure. $\square$

Corollary 4.5 (No General Grounding). *There is no environment-independent grounding coefficient. An agent grounded in one domain is 5^+ in that domain and potentially 5^* in another.*

5 Loop Closure and the Grounded Data Flywheel

Definition 5.1 (Loop Closure). Agent $\mathcal{A}$ *closes the loop* in environment $\mathcal{E}$ if it has a self-sourced observation operator $\omega_{\mathcal{E}}$ with respect to $\mathcal{E}$ and $G_{\mathcal{E}}(\mathcal{A}) > 0$.

Definition 5.2 (Grounded Data). The *grounded data* of agent $\mathcal{A}$ in environment $\mathcal{E}$ is

$$\mathcal{D}_{\mathcal{A}}(\mathcal{E}) = \bigcup_{s \in \mathcal{S}_{\mathcal{E}}} \mathcal{B}_{\mathcal{A}}(s) \setminus \nu(\mathcal{C}_{\text{public}})$$

where ν ranges over all derivation maps from public corpora $\mathcal{C}_{\text{public}}$.

Theorem 5.3 (Flywheel Non-Derivability). *If $\mathcal{A}$ closes the loop in $\mathcal{E}$, then $\mathcal{D}_{\mathcal{A}}(\mathcal{E}) \neq \emptyset$.*

Proof. Suppose $\mathcal{D}_{\mathcal{A}}(\mathcal{E}) = \emptyset$. Then every content in $\mathcal{B}_{\mathcal{A}}(s)$ for $s \in \mathcal{S}_{\mathcal{E}}$ is derivable from public corpora. Hence $G_{\mathcal{E}}(\mathcal{A}) = 0$, contradicting loop closure (Definition 5.1). $\square$

Corollary 5.4 (Grounding Cannot Be Purchased). *Let $\mathcal{B}$ be an agent that has not interacted with $\mathcal{E}$. Then no purchase of components by $\mathcal{B}$ yields $G_{\mathcal{E}}(\mathcal{B}) > 0$ without $\mathcal{B}$ ’s own interaction with $\mathcal{E}$.*

Proof. By Theorem 3.4, purchase of generative components does not change $G_{\mathcal{E}}$. Purchase of transductive components provides the hardware for interaction but not the interaction itself. By Theorem 5.3, grounding requires self-sourced states. Hence acquisition requires $\mathcal{B}$ ’s own interaction with $\mathcal{E}$. $\square$

6 The Integrator Moat

Definition 6.1 (Integrator). An *integrator* is an agent (or firm) that closes the loop in some environment $\mathcal{E}$.

Theorem 6.2 (Integrator Moat). *An integrator that closes the loop in $\mathcal{E}$ acquires a resource $\mathcal{D}_{\mathcal{A}}(\mathcal{E})$ that is not obtainable by component purchase.*

Proof. By Theorem 5.3, $\mathcal{D}_{\mathcal{A}}(\mathcal{E}) \neq \emptyset$. By Corollary 5.4, this data cannot be acquired by component purchase. Hence it constitutes a resource unavailable to non-integrators. $\square$

Corollary 6.3 (The Moat Is the Loop). *The proprietary asset of an integrator is not the model, the orchestrator, or the body, but the closed loop that generates grounded data.*

Remark 6.4. Component vendors may hold moats of their own—compute, manufacturing, network effects. The theorem claims only that the loop is a moat, not that it is the only moat.

7 Predictions for Industrial Structure

Prediction 7.1 (Component Commoditization). Generative components (model, orchestration) commoditize: their substitution changes R and D but not G (Corollary 3.5). Transductive components (sensors, actuators) also commoditize as manufacturing scales. The integrator position is not a component.

Prediction 7.2 (Domain-Specific Integrators). By Theorem 4.4, grounding is environment-specific. Hence the integrator landscape consists of domain-specific integrators, not one general winner.

Prediction 7.3 (Value Accrues to the Loop). The proprietary asset of an integrator is $\mathcal{D}_{\mathcal{A}}(\mathcal{E})$ (Theorem 6.2). Hence value accrues to firms that close the loop, not to firms that supply components.

Prediction 7.4 (Flywheel Dynamics). An integrator that closes the loop improves its grounding over time via the epistemic loop, generating new grounded data as a byproduct. Competitors must build their own loop and live their own experience to replicate this.

7.1 Failure Modes

We record three ways the predictions could fail. They are recorded here because the strength of the framework’s claims makes it important to state their defeat conditions.

Remark 7.5 (Integration Standardization). If loop-closure architecture standardizes—the epistemic loop becomes a reusable framework, like TCP/IP—then integration itself commoditizes. Value would then migrate to whoever owns deployment channels or grounded data at scale.

Remark 7.6 (Component Non-Commoditization). If generative or transductive components retain durable moats (compute, manufacturing, network effects), the integrator thesis weakens. Theorem 6.2 would still hold, but the moat might be dominated by a component moat.

Remark 7.7 (Hardness of Grounding). If loop closure in unstructured environments proves harder than current demonstrations suggest, the timeline stretches and the qualitative transition remains out of reach longer than the predictions suggest.

8 The Integrator Position

Theorem 8.1 (Integrator Position Is Structural). *The integrator’s advantage is not a temporary market position but a structural consequence of Definitions 5.1–6.1 and Theorems 5.3–6.2. It arises wherever grounding is environment-specific and cannot be derived from public corpora.*

Proof. Follows from Theorem 4.4 (environment-specificity), Theorem 5.3 (non-derivability), and Theorem 6.2 (non-purchasability). Together these establish that the integrator acquires a resource structurally unavailable to non-integrators. $\square$

Corollary 8.2 (The Future Looks Boring). *The most valuable firms in the grounded-agent economy may look like integrators—commodity components wired into a loop that lives in the world—rather than like model labs, robotics vendors, or orchestration platforms.*

Remark 8.3. This mirrors the computing industry: the highest-value firms did not make CPUs or operating systems, but integrated components into products that closed a loop with users.

9 The Tinkering Threshold and the PC-Era Analogy

The previous sections answer *where* value accrues in the production of grounded agents. They do not answer *when* the production of such agents becomes cheap enough that individual tinkerers, not laboratories, become the primary innovators. We formalize this question and show that the robotics trajectory differs structurally from the PC trajectory.

9.1 Cost Curves

Definition 9.1 (Component Cost Curve). Let $\mathcal{K}$ be a class of components. Its *cost curve* is a function $c_{\mathcal{K}} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ giving the unit cost of a component in $\mathcal{K}$ at time t . Component cost curves are typically *decaying*: $c_{\mathcal{K}}(t) \rightarrow 0$ as $t \rightarrow \infty$, at some rate $\lambda_{\mathcal{K}} > 0$.

Definition 9.2 (Integration Cost). Let $\mathcal{A}$ be an agent assembled from components $\kappa_1, \dots, \kappa_n$. The *integration cost* of $\mathcal{A}$ at time t , written $c_{\text{int}}(t)$, is the cost of wiring the components into a closed epistemic loop (Definition 5.1): configuring interfaces, calibrating sensors, connecting the model-generator to the observation operator, and verifying loop closure.

Definition 9.3 (Total System Cost). The *total system cost* of a prototype agent $\mathcal{A}$ at time t is

$$c_{\text{tot}}(\mathcal{A}, t) = \sum_{i=1}^n c_{\kappa_i}(t) + c_{\text{int}}(t).$$

Definition 9.4 (Tinkering Threshold). The *tinkering threshold* τ_{tink} is the maximum total system cost that an individual hobbyist will bear for a kit. We treat τ_{tink} as roughly constant in real terms across eras, on the order of a few thousand dollars at present value.

Definition 9.5 (Tinkering Arrival Time). The *tinkering arrival time* of a system class $\mathcal{S}$ is

$$t^*(\mathcal{S}) = \inf\{t : c_{\text{tot}}(\mathcal{A}, t) \leq \tau_{\text{tink}} \text{ for some } \mathcal{A} \in \mathcal{S}\}.$$

9.2 The Decay Divergence

Theorem 9.6 (Integration Cost Decays Slower than Component Cost). *There exist component classes $\mathcal{K}$ and system classes $\mathcal{S}$ such that $c_{\mathcal{K}}(t)$ decays exponentially while $c_{\text{int}}(t)$ for $\mathcal{S}$ decays sub-exponentially or not at all.*

Proof. By Definition 9.1, component cost is governed by manufacturing scale: $c_{\mathcal{K}}(t) = c_{\mathcal{K}}(0)e^{-\lambda_{\mathcal{K}}t}$. Integration cost is governed by the standardization of the loop architecture, not by manufacturing. If loop closure requires domain-specific epistemic work—calibration, falsification, revision specific to an environment—then its cost is not a function of production volume. Hence it does not share the component curve. $\square$

Corollary 9.7 (Component Cheapness Is Not Sufficient). *A system class may have arbitrarily cheap components and still fail to reach the tinkering threshold, if integration cost remains above $\tau_{\text{tink}} - \sum_i c_{\kappa_i}(t)$.*

This is the formal statement that cheap parts do not make a cheap loop.

9.3 Grounding Externalization and Internalization

The PC era and the robotics era differ in one structural respect: where the grounding lives.

Definition 9.8 (Externalized Grounding). An agent $\mathcal{A}$ has *externalized grounding* if $G(\mathcal{A}) = 0$ but $\mathcal{A}$ is embedded in a loop whose grounding is supplied by another agent $\mathcal{B}$ with $G(\mathcal{B}) > 0$.

Definition 9.9 (Internalized Grounding). An agent $\mathcal{A}$ has *internalized grounding* if $G(\mathcal{A}) > 0$ by its own self-sourced observation.

Proposition 9.10 (PCs Externalize, Robots Internalize). *A personal computer is an externalized-grounding agent. A general-purpose autonomous robot is an internalized-grounding agent.*

Proof. A personal computer has no self-sourced observation operator (its sensory states are user-mediated) and no autonomous action in a physical environment. Its utility is realized only within a human loop that supplies grounding. By Definition 9.8, it is externalized. A general-purpose autonomous robot must operate without continuous human grounding; it must test its models against its own sensors. By Definition 9.9, it is internalized. $\square$

Theorem 9.11 (Externalization Lowers the Cost Curve). *If an agent class has externalized grounding, its total system cost is governed by component cost alone:*

$$c_{\text{tot}}(\mathcal{A}, t) = \sum_{i=1}^n c_{\kappa_i}(t).$$

Hence its tinkering arrival time is governed by the fastest-decaying component.

Proof. Externalized grounding implies that loop closure is not the agent’s responsibility. Integration cost is therefore negligible relative to component cost, and the total cost reduces to the component sum. By Definition 9.1, this decays exponentially. $\square$

Corollary 9.12 (PC-Era Rapidity). *The PC era arrived rapidly because PCs externalize grounding. Their cost curve was the component curve.*

Corollary 9.13 (Robotics-Era Delay). *The robotics era is delayed relative to the component curve because robots internalize grounding. Their cost curve is the slower of the component curve and the loop-standardization curve.*

9.4 Multi-Environment Generality

The final structural difference concerns the number of environments over which generality must hold.

Definition 9.14 (Multi-Environment Grounding Requirement). A *general-purpose* agent with respect to environments $\mathcal{E}_1, \dots, \mathcal{E}_n$ must satisfy $G_{\mathcal{E}_i}(\mathcal{A}) > 0$ for all $i \in \{1, \dots, n\}$.

Theorem 9.15 (Generality Delays the Tinkering Threshold). *Let $\mathcal{A}$ be an agent. Then*

$$t^*(\mathcal{A} \text{ general-purpose}) \geq \max_i t^*(\mathcal{A} \text{ specialized to } \mathcal{E}_i).$$

Moreover, the inequality is strict whenever the environments do not share sensory modality structure.

Proof. By Theorem 4.4, grounding is environment-specific. Acquiring $G_{\mathcal{E}_i}(\mathcal{A}) > 0$ for each i requires interaction with $\mathcal{E}_i$. If environments do not share modality structure, the grounding acquired in one does not transfer to another, so grounding must be acquired sequentially. Hence the general-purpose arrival time is bounded below by the maximum specialized arrival time, and is strictly greater when transfer is absent. $\square$

Corollary 9.16 (Staggered Eras). *The robotics era will not be a single event but a sequence of domain-specific eras: specialized robots reach the tinkering threshold before general-purpose robots do, and each domain has its own arrival time.*

9.5 Predictions

Prediction 9.17 (Phases of the Robotics Era). Let τ_{tink} be the tinkering threshold. Then:

- (1) **Phase 1 (mainframe era, present).** $c_{\text{tot}} \gg \tau_{\text{tink}}$. Integration labor dominates. Adopters are labs and well-funded firms.
- (2) **Phase 2 (minicomputer era, 3–7 years).** Component costs fall to \$3k–\$8k; on-device inference eliminates cloud costs; open-source orchestration standardizes. c_{int} drops to \$5k–\$15k. $c_{\text{tot}} \approx \$10k - -\$25k$. Accessible to serious hobbyists and small businesses.
- (3) **Phase 3 (PC era, 7–12 years, specialized domains).** Humanoid platforms at \$1k–\$3k; on-device inference near-free; orchestration standardized. $c_{\text{int}} \rightarrow 0$ in domains with standardized loops. $c_{\text{tot}} \approx \$1.5k - -\$4k$. Accessible to individual developers.
- (4) **Phase 4 (smartphone era, 12–20 years, general-purpose).** General-purpose robots reach τ_{tink} . Requires loop standardization across multiple environments, which is bounded below by Theorem 9.15.

Prediction 9.18 (Specialized Before General). Specialized robots (warehouse, cleaning, cooking) will reach PC-era prices before general-purpose robots. This follows from Theorem 9.15.

Prediction 9.19 (Loop Standardization Is the Bottleneck). The limiter on the general-purpose robotics era is not component cost, but loop standardization. Firms that make the loop easy to assemble—not those that make components cheapest—will launch the era.

9.6 How the Robotics Trajectory Differs from the PC Trajectory

We summarize the structural differences in four propositions, each following from results above.

Proposition 9.20 (Integration, Not Interface). *PC integration was an interface problem, solved once by bus and port standardization. Robot integration is an epistemic problem—loop closure—solved only per environment. Hence robot integration cost decays slower than component cost (Theorem 9.6), while PC integration cost collapsed to negligible levels.*

Proposition 9.21 (Externalization vs. Internalization). *PCs externalize grounding to the human user (Proposition 9.10), so their cost curve is the component curve (Theorem 9.11). Robots internalize grounding, so their cost curve includes the loop-standardization curve, which is the slower of the two.*

Proposition 9.22 (Combinatorial vs. Environmental Universality). *PC universality is combinatorial over software: any program runs on the same hardware. Robot generality is environmental: each environment requires grounding (Definition 9.14). The first scales with code; the second scales with lived interaction, which does not transfer without shared modality structure.*

Proposition 9.23 (One Era vs. Staggered Eras). *The PC era was a single event. The robotics era will be staggered (Corollary 9.16): domain-specific PC eras arrive at different times, and the general-purpose era is bounded below by the maximum of the specialized ones (Theorem 9.15).*

Remark 9.24 (The PC Analogy Holds Locally, Not Globally). The PC analogy holds for each specialized domain: a warehouse robot can have a PC-like cost curve once its loop is standardized. The analogy fails for general-purpose robotics, where the grounding requirement is per-environment and cannot be componentized.

Corollary 9.25 (The Right Comparison). *The PC era is not the right analogy for general-purpose robotics. The right analogy is a federation of domain-specific eras, each with its own tinkering arrival time and its own integrator moat (Theorem 6.2).*

10 Open Problems

Open Problem 10.1. What is the rate of loop standardization? Can it be measured, or only observed post hoc? If measurable, it would give an empirical estimate of the robotics-era arrival times in Prediction 9.17.

Open Problem 10.2. Must the integrator of an environment be an original loop-closer, or can a second integrator acquire $G_{\mathcal{E}} > 0$ by copying the first’s architecture and living its own experience more cheaply? The framework (Theorem 6.2, Corollary 5.4) says grounding cannot be purchased, but it does not bound the cost of *living* it.

Open Problem 10.3. Can accidental fitness (Definition 3.16) be predicted from component and workload structure alone, or only observed post hoc? A predictive theory would allow identification of future Nvidia-like firms before they occupy the adjacent component market.

Open Problem 10.4. Is there a formal characterization of $c_{\text{int}}(t)$ — the cost of closing a loop in domain $\mathcal{E}$ — that separates the standardization component (shared across domains) from the epistemic component (specific to $\mathcal{E}$)? This is the measurement problem behind Prediction 9.19.

Remark 10.5. The predictions of this paper are consequences of the axioms of Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*, not forecasts about engineering timelines or market outcomes. They state what follows *if* an agent or firm has the stated properties, not when or whether such agents or firms will exist.

11 Conclusion

We have applied the inference-first framework to industrial structure and to consumer adoption. The central results are:

1. Generative components commute with G : swapping model or orchestration changes R and D but not grounding (Theorem 3.4). Hence scale does not substitute for grounding.
2. Commoditization is driven by interface standardization, not by specification convergence (Proposition 3.9). Spec sheets are downstream of interfaces.
3. A firm dominating a commodity specialization for one workload can capture the component layer of an adjacent workload via accidental fitness (Theorem 3.17).
4. Grounding is environment-indexed (Theorem 4.4); it cannot be transferred across domains without interaction.
5. Grounded data is non-derivable from public corpora (Theorem 5.3) and non-purchasable (Corollary 5.4). The integrator who closes the loop acquires a structural moat (Theorem 6.2).
6. The integrator position is structural, not temporary (Theorem 8.1); the predicted landscape is one of domain-specific integrators rather than a single general winner.
7. Integration cost decays slower than component cost (Theorem 9.6). Cheap components are not sufficient for a cheap loop.
8. PCs externalize grounding to the user (Proposition 9.10), so their cost curve is the component curve (Theorem 9.11). Robots internalize grounding, so their cost curve includes the loop-standardization curve.
9. Generality requires per-environment grounding (Definition 9.14), so the general-purpose tinkering threshold is bounded below by the maximum of the specialized thresholds (Theorem 9.15).
10. The robotics era is therefore not a single event but a sequence of domain-specific eras (Corollary 9.16). The PC analogy holds locally (per domain) and fails globally (Remark 9.24).
11. Loop standardization is the bottleneck, not component cost (Prediction 9.19).

Applied to the present moment, the framework yields a sharp prediction: model, orchestration, and hardware layers commoditize; the closed loop does not. The proprietary asset is the grounded data generated by lived interaction in a specific environment. The firms that close the loop first acquire a resource that competitors cannot buy and cannot derive. The predicted landscape is not one winner but many, each grounded in a distinct domain, and the most valuable firms may look boring—integrators, not labs. This is the industrial content of the claim that the grounded-agent economy is a federation, not a single computation.

A Summary of Definitions (Application-Specific and Imported)

Symbol	Name	Reference
κ	Component	Definition 3.1
$\mathcal{A}[\kappa \rightarrow \kappa']$	Substitution	Definition 3.2
$I(\mathcal{K})$	Interface	Definition 3.6
Interface standard	Common interface across class	Definition 3.7
Performance parity	Substitution preserves R, D	Definition 3.8
W	Workload class	Definition 3.12
$P(\kappa)$	Design profile	Definition 3.13
$F(\kappa, W)$	Substrate fitness	Definition 3.14
Commodity specialization	Industrial-scale, single-workload component	Definition 3.15
Accidental fit	High fitness for non-design workload	Definition 3.16
$\mathcal{E}$	Environment	Definition 4.1
$G_{\mathcal{E}}(\mathcal{A})$	Environment-indexed grounding	Definition 4.2
$\mathcal{D}_{\mathcal{A}}(\mathcal{E})$	Grounded data	Definition 5.2
Loop closure	$G_{\mathcal{E}}(\mathcal{A}) > 0$	Definition 5.1
Integrator	Loop-closing firm	Definition 6.1
$c_{\mathcal{K}}(t)$	Component cost curve	Definition 9.1
$c_{\text{int}}(t)$	Integration cost	Definition 9.2
$c_{\text{tot}}(\mathcal{A}, t)$	Total system cost	Definition 9.3
τ_{tink}	Tinkering threshold	Definition 9.4
$t^*(\mathcal{S})$	Tinkering arrival time	Definition 9.5
Externalized grounding	$G = 0$ embedded in another's loop	Definition 9.8
Internalized grounding	$G > 0$ by own observation	Definition 9.9
Multi-environment requirement	$G_{\mathcal{E}_i} > 0$ for all i	Definition 9.14

B Summary of Theorems (Application-Specific and Imported)

Theorem	Statement
3.4	Generative substitution invariant under G
3.5	Model layer commoditizes
3.9	Interfaces drive commoditization
3.10	Spec sheets are downstream
3.17	Accidental fit produces outsider dominance
3.18	Outsider unpredictability
3.19	The Nvidia case
4.4	Grounding is environment-specific
4.5	No environment-independent grounding
5.3	Grounded data is non-derivable
5.4	Grounding cannot be purchased
6.2	Integrator moat
6.3	The moat is the loop
8.1	Integrator position is structural
8.2	The future looks boring
9.6	Integration cost decays slower than component cost
9.7	Cheap components are not sufficient for a cheap loop
9.10	PCs externalize; robots internalize
9.11	Externalization lowers the cost curve
9.12	PC-era rapidity explained
9.13	Robotics-era delay explained
9.15	Generality delays the tinkering threshold
9.16	Staggered domain-specific eras

C Industrial-Structure Level Table

Firm type	Provides	R	D	G
Model lab	generative component	high	high	≈ 0
Orchestration platform	generative component	—	—	≈ 0
Robotics vendor	transductive component	—	—	≈ 0
Domain integrator	closed loop	high	high	> 0

Remark C.1. The “integrator” row is the only one with $G > 0$, and hence the only one that crosses the $5^* \rightarrow 5^+$ threshold. This is the formal content of Corollary 8.2.

D PC-Era vs. Robotics-Era Comparison

Dimension	PC era	Robotics era
Grounding	Externalized to user	Internalized by agent
Integration	Interface (bus, port, driver)	Epistemic (loop closure)
Lagging variable	Component cost	Loop standardization
Universality	Combinatorial over software	Per-environment grounding
Number of eras	One	Many (staggered)
Tinkering arrival	Single date (mid-1970s)	Sequence of domain-specific dates
Winning position	Component integrator (PC OEM)	Loop integrator (domain-specific)

Remark D.1. Each row corresponds to a proposition or theorem in Section 9. The table is the compact form of the claim that the PC analogy holds locally, not globally (Remark 9.24).

E A Note on Method

This paper proceeds by applying the formal framework of Paper 0, *The Inference-First Picture of Mind, Logic, and Mathematics*. Its claims are conditional and framework-relative, and its predictions are consequences of the axioms, not forecasts about engineering timelines or market outcomes.